K-means Clustering (from scratch) report

Shefali Maurya

Cyber Analytics

sm9991@rite.edu

***Abstract*— K-means clustering is a machine learning algorithm that groups unlabeled data into a specific number of distinct, special clusters. This could uncover hidden groups or patterns that might not be obvious.**

***Keywords*—** k-mean, machine learning, euclidean, mahalanobis

I. Problem & dataset

K-mean clustering requires a way to calculate distance between two values. There are two ways I will look at it: Euclidean distance, which measures the straight-line distance between two points, and Mahalanobis Distance, which measures how many standard deviations away a point is from the mean of a cluster.

While Python *scklearn* has a method to execute the Euclidean distance algorithm, the Mahalanobis distance algorithm requires a manual implementation. For this project, I will implement a manual implementation of the Euclid and Mahalanobis algorithm and compare the results against the built in Euclidean method in *scklearn.*

In order to do this, I will utilize the UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems paper for this, which has 1 normal and 9 attack feature group classes. Each run will give a visual result for the data and clusters, a confusion matrix, k selection using the elbow method and Silhouette (euclidean geometry), and metrics for each mahalanobis algorithm and euclidean algorithm.

II. Implementation

I implemented Lloyd's algorithm with support for Euclidean and diagonal Mahalanobis distances.

A. K-means initialization

My implementation supports both random and k-means++ initialization.

For **Random initialization,** k distinct samples are picked uniformly at random.

For **k-means++** initialization

- The first centroid is picked uniformly
- for each centroid after that
  - Each points squared distance to its nearest chosen centroid is chosen
  - Sample the next centroid with probability proportional to that distance

B. K-means Assignment

Each point is assigned to the nearest centroid by computing the squared distances under the configured metric (Euclidean or Mahalanobis)

C. K-means Update

Each centroid is recomputed as the mean of its assigned points.

An **empty cluster** is handled by leaving the centroid where it is.

D. K-means Distance Metrics

My implementation supports both random and k-means++ initialization. As stated before, the model implants both Euclidean and Mahalanobis distance.

The Euclidean computation is a squared calculation of each point distance to every centroid.

The Mahalanobis computation is a squared calculation that is a bit more complicated.

The equation is:

$$D^2 = (x - m)^T * C^{-1} * (x - m)$$

where D^2 is the mahalanobis distance, x is the vector in the observation row, m is the vector of mean values of independent variables, and C^{-1} is the inverse covariance matrix.

E. K-means Fit Loop (Lloyds Algorithm and Restarts)

The algorithm requires restarting multiple time to refine the values in an iterative manner. It uses all the steps mentioned above

- Assignment
- Update
- Convergence if
  - labels stop changing

- centroid movement is less than the total

At the end, the final attributes - centroids, labels, inertia, number of iterations - are computed.

III. CHOOSE K

When running the model, both k sweeps for Euclidean and Mahalanobis state k=6 has the best silhouette. So I chose 6 for k.

IV. *Evaluation*

Ground truth labels were used only for the permitted sampling design and for external evaluation, not for clustering or centroid updates.

A. External (label-based) evaluation

From the run I am basing this report on, the cluster -> class majority mapping was

{0→5, 1→6, 2→4, 3→6, 4→2, 5→1}

The confusion matrix shows that the model clusters capture coarse traffic modes rather than fine-grained attack categories. Several attack types (such as classes 5 and 6) dominate multiple clusters while others appear sparsely.

The label-based metrics for this run:

Accuracy: 0.3158
Precision: 0.3101
Recall: 0.2879,
F1: 0.2156
ARI: 0.1013
NMI / **V-measure**: 0.2634

These values indicate moderate alignment between clusters and attack categories

B. Internal (geometry-only) evaluation

Internal metrics reflect the structure of the clusters independent of labels:

Silhouette: 0.4278
Calinski–Harabasz: 1652.11
Davies–Bouldin: 1.5441

When comparing these metrics against other k values, it shows better separation and more compact clusters.

V. *Euclidean vs. Mahalanobis*

Both geometries use the exact 39-feature order:

["dur","spkts","dpkts","sbytes","dbytes","rate","sttl","dttl","sload","dload","sloss","dloss","sinpkt","dinpkt","sjit","djit","swin","stcpb","dtcpb","dwin","tcprtt","synack","ackdat","smean","dmean","trans_depth","response_body_len","ct_srv_src","ct_state_ttl","ct_dst_ltm","ct_src_dport_ltm","ct_dst_sport_ltm","ct_dst_src_ltm","is_ftp_login","ct_ftp_cmd","ct_flw_http_mthd","ct_src_ltm","ct_srv_dst","is_sm_ips_ports"]

Both of them have the same parameters to make sure that the comparisons are fair

A. Mahalanobis diagonal C^{-1}

Mahalanobis run uses a diagonal covariance approximation. This rescales features by variance, where High variance features get weight 2.0, and Low variance features get weight 0.5. This changes the geometry by stretching and compressing axes.

B. Comparing silhouette + label metrics

Internal metrics (Euclidean → Mahalanobis):

- ***Silhouette**: 0.4278 → 0.4254 (euclidean geometry)*
- ***Silhouette** (configured Mahalanobis): 0.4323 (slightly higher)*
- ***CH**: 1652.11 → 1872.30*
- ***DB**: 1.5441 → 1.2989 (lower is better)*
-

External metrics (Euclidean → Mahalanobis):

- ***Accuracy**: 0.3158 → **0.3407***
- ***F1**: 0.2156 → **0.2380***
- ***ARI**: 0.1013 → **0.1342***
- ***NMI**: 0.2634 → **0.3138***

The Diagonal Mahalanobis performs only a little better than Euclidean at k=6.

This might be because Mahalanobis down weights noisy low variance features (0.5 enteries) and up weights informative high variance features (2.0 entries). This reshaping improves cluster separation and label alignment.

VI. *Conclusion*

From the run, the k-sweep shows that k = 6 is the best choice based on elbow flattening and silhouette peak. Compared to Euclidean clustering, Diagonal Mahalanobis geometry yields modest but consistent improvements in both internal and external metrics.

VII. Limitations

K-means assumes spherical clusters, which is unrealistic for real world data. In addition, UNSW-NB15 contains highly overlapping classes, making perfect separation impossible for K-means.